

```
$: rm -rf *
$: cd ../w32api
$: ./configure --prefix=/opt/mingw/686-pc-mingw32 --host=i686-pc-mingw32
$: make && sudo make install
$: cd ../mingwrt-3.15.2-mingw
$: ./configure --prefix=/opt/mingw/686-pc-mingw32 --host=i686-pc-mingw32
$: make && sudo make install
$: cd ../build
$: sudo ./gcc-4.4.0/configure --prefix=/opt/mingw --target=i686-pc-mingw32 \
  --with-headers=/opt/mingw/686-pc-mingw32/include \
  --disable-shared --enable-languages=c,c++,fortran
$: sudo make
$: sudo make install
```

That's it! You have a fully working C, C++ and Formula Translator compiler toolchain that'll compile code meant to run on Windows!

More

MinGW does not provide a POSIX implementation, so you are out of luck on compiling programs that rely on POSIX functions. Not many do, and most that do, have an alternative set of sources meant to use the Win32API to replace POSIX calls by native Win32 ones. But once in a while, if you need POSIX API support, Google for Cygwin and download what's required. Beware though, Cygwin doesn't come as a cross-compiler and it's terribly difficult to build one. Cygwin requires Windows NT 5 and above to run.

(NT 5 is Windows 2000. XP is NT 5.1, and Windows Server 2003 is NT 5.2, as is Windows XP Professional x64 Edition. Vista and Server 2008 are NT 6, and Windows 7 and Server 2008 R2 are NT 6.1. This information will help when you want to develop a program meant to run only on certain versions of Windows, as internally, all Windows OSs 'know' themselves by their NT version numbers and not their names.)

But hey...

...we just built a cross-compiler, so how do we use it? It can be made difficult, and then it can be made simple. Actually, all you need are some environment variables. Here goes:

The first step is adding the path `/opt/mingw/bin` to your PATH variable. Type in the export command that we executed after building Binutils.

The second step is configuring the source with a `--host=i686-pc-mingw32` flag.

That's all.

There's one final thing to do before signing off: testing the GCC compilers. I don't know any Fortran, so I could never test Fortran; however quite a lot of GNU Projects insist on having some version of Fortran in the toolchain, so it's a safe bet to keep it built. We need to test the compilers in a "Hello World" program. For C, you can use the one in the example `src.c` file in the previous page; for C++ the program goes somewhat like:

```
// File: src.cc
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello World!" << endl;
    return 0;
}
```

Compile both the files with:

```
$: i686-pc-mingw32-gcc src.c -o c.exe
$: i686-pc-mingw32-g++ src.cc -o cxx.exe
```

Here are the results of a comprehensive testing of both Binutils and GCC Components. Oh wait, I exaggerated :-):

```
bg14ina@bg14ina-desktop:~/Desktop/src$ ls -l
total 3896
-rwxr-xr-x 1 bg14ina bg14ina 27363 2009-06-11 16:02 c.exe
-rwxr-xr-x 1 bg14ina bg14ina 3952542 2009-06-11 16:02 cxx.exe
-rw-r--r-- 1 bg14ina bg14ina 73 2009-06-07 16:02 src.c
-rw-r--r-- 1 bg14ina bg14ina 105 2009-06-11 16:01 src.cc

bg14ina@bg14ina-desktop:~/Desktop/src$ file c.exe
c.exe: PE32 executable for MS Windows (console) Intel 80386 32-bit
bg14ina@bg14ina-desktop:~/Desktop/src$ file cxx.exe
cxx.exe: PE32 executable for MS Windows (console) Intel 80386 32-bit

bg14ina@bg14ina-desktop:~/Desktop/src$ i686-pc-mingw32-strip \
> -s (c,cxx).exe

bg14ina@bg14ina-desktop:~/Desktop/src$ ls -l
total 528
-rwxr-xr-x 1 bg14ina bg14ina 4096 2009-06-11 16:16 c.exe
-rwxr-xr-x 1 bg14ina bg14ina 525312 2009-06-11 16:16 cxx.exe
-rw-r--r-- 1 bg14ina bg14ina 73 2009-06-07 16:02 src.c
-rw-r--r-- 1 bg14ina bg14ina 105 2009-06-11 16:01 src.cc

bg14ina@bg14ina-desktop:~/Desktop/src$ wine c.exe
Hello World!
bg14ina@bg14ina-desktop:~/Desktop/src$ wine cxx.exe
Hello World!
```

There you go! One boxed product, ready to run! Now I gotta go build myself a version of libVLC. Bye.. 

By: Boudhayan Gupta

Boudhayan is a 14-year old student who suffers from an acute psychological disorder called Distromania. He owes his life to Larry Page and Sergei Brin. Apart from that, he enjoys both reading and writing, and when he is not playing with his Python :-), during most of his spare time he can be found listening to Fort Minor, or cooking.